

in Map objects. This is useful in finding out if searches can be optimised by choosing a better hash code algorithm for the keys or by setting a different load factor for these maps.

Prior to Java 1.7, *substring()* implementation in *java.lang.String* returned a string object that was still backed by the original *char* array, but with a different offset and starting pointer. This was done to make *substring()* method faster. But this resulted in a memory leak because even though an original large string was eligible for garbage collection, its underlying huge *char* array could not be garbage collected even if there was one small sub-string object created. This wastage can be viewed in the 'Waste in *char* arrays' feature.



Note: Since Java 7, the sub-string method of string returns a new string that does not point to the original string's *char* array. This is a trade-off in favour of decreased memory usage over increased time to create sub-strings.

In addition to the tools described above, MAT also has an OQL console, where specific queries can be entered and executed. The OQL console can be launched by clicking the OQL button. The exact syntax of the OQL commands can be obtained by hitting F1 in the OQL console.

IBM Thread Analyser

IBM Thread and Monitor Dump Analyser for Java can be downloaded from the IBM developer works site: <https://www.ibm.com/developerworks/community/groups/service/html/communityview?communityUid=2245aa39-fa5c-4475-b891-14c205f7333c>

This is in the form of an executable *jar* file. It can be used to analyse thread dumps created by IBM JVMs. As mentioned in the introduction, thread and heap dumps for an IBM JVM can be triggered by sending a *Control+Break* signal to the JVM. Once the dump is generated, the core file, usually with the name *javacore.<timestamp>.txt* can be opened in the IBM Thread Analyser. Like JVisual VM's thread analyser, the IBM Thread Analyser also detects any deadlocks in the running application and reports them when the dump is loaded.

When a dump file is loaded, the Java system properties and environment variables are listed along with the thread details. In addition, a heap usage summary at the point where the dump was taken and information on previous garbage collection cycles is given. Most of the useful features are available in the 'Analysis' menu option.

The basic analysis that can be done is Thread Status Analysis. This gives a graphical view of the threads, based on their states. The details of those threads are displayed in the bottom panel. For each thread, the name, state, native thread ID, the Java method and stack depth are displayed. Individual threads from this view can be selected and details about them are displayed on the right hand side. The Java stack traces of a thread, along with the monitor that it owns and the monitor it is waiting on, are displayed. This is useful in identifying

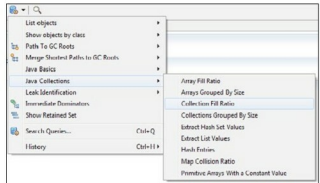


Figure 7: MAT Java collections

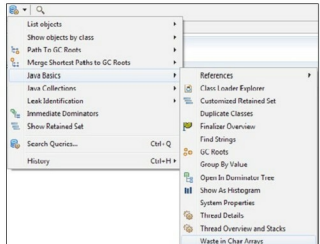


Figure 8: MAT Java basics arrays

the piece of code that has caused a deadlock (if any) and the monitor that is needed to break a deadlock.

The tool also distinguishes between threads that are waiting on a monitor and threads that are waiting on a condition. A thread that is waiting to acquire a lock is in **BLOCKED** or *waiting on lock* condition. A thread that is waiting to be notified, usually via *notify()* or *notifyAll()*, of some condition so that execution can proceed will be in **WAITING** or *wait on condition* state. In general, threads that are in **BLOCKED** state are a cause of concern as they indicate a *lock* condition. Threads in **WAITING** state may or may not be a cause of concern based on which monitor's condition they are waiting on. There can be overlaps in thread states. For instance, a blocked thread could also have been deadlocked. In this case, the status is displayed as *Deadlock/Blocked*. The percentage of threads in each state, displayed in Thread Status Analysis, could exceed 100 per cent because threads in dual conditions, as described above, are reported under both states.

The Method Analysis view organises the threads based on their status and the Java method they are in. No Java stack can be reported in case of the native code.

A very useful feature in analysing the performance of applications is the ability to compare two thread dumps. By comparing the thread dumps taken at different points in